

Multi-Layer Perceptron: Why Stacking Perceptrons Bends the Boundary

Contents

1. Overview	2
2. One perceptron with a sigmoid	2
3. Combining two perceptrons	3
3.1 The idea: superimpose, then smooth	3
3.2 Adding the probabilities	3
3.3 Weights and a bias	4
3.4 The combiner is a perceptron	4
4. Four ways to change the architecture	5
4.1 More nodes in a hidden layer	6
4.2 More nodes in the input layer	6
4.3 More nodes in the output layer	7
4.4 More hidden layers	7
5. Seeing it work	7
5.1 XOR with two hidden nodes	7
5.2 Other shapes	9
6. Summary	11
7. Sources	11
8. Key terms	11

Where this fits: Pipeline map steps 0 (Foundations), 8 (Model). See the [Course map](#).



- **Builds on:** Sigmoid function ([Note 74](#)); Neural networks ([Note 1002](#)); Perceptron ([Note 1004](#)); Problem with the perceptron (XOR) ([Note 1007](#)); MLP notation and parameter count ([Note 1008](#)).
- **Leads to:** Keras workflow ([Note 1011](#)); ANN for classification ([Note 1011](#)).

1. Overview

Key point: One perceptron draws one straight line. Feeding the outputs of several perceptrons into another perceptron combines their lines into a curved boundary, and that combination is a multi-layer perceptron.

A single perceptron cannot separate data whose classes need a curved boundary, such as XOR (see the [problem with the perceptron Note](#)). The fix must be built from perceptrons themselves. This Note builds the **multi-layer perceptron (MLP)** from first principles: two perceptrons, one way to combine them, and then larger networks.

Throughout this Note every perceptron uses the **sigmoid** activation, not the step function. So each perceptron outputs a probability between 0 and 1, exactly like logistic regression.

2. One perceptron with a sigmoid

Key point: A perceptron with a sigmoid gives every point a probability: 0.5 on its line, rising towards 1 on one side and falling towards 0 on the other.

With the sigmoid as activation and binary cross-entropy as loss, a perceptron is logistic regression (see the [perceptron loss Note](#)). Each student is one **observation** (one record, one row of the data table). CGPA x_1 and IQ x_2 are its two **features** (input variables, one column each), and placed or not is the **target** (the output we predict). For one student the perceptron computes

$$\hat{y} = \sigma(w_1x_1 + w_2x_2 + b),$$

the probability that the student is placed. The probability of not being placed is $1 - \hat{y}$.

The [sigmoid function Note](#) (section 5.2) draws this as a map. On the line $w_1x_1 + w_2x_2 + b = 0$ the probability is 0.5. Lines parallel to it carry 0.6, 0.7, 0.8, ... on one side and 0.4, 0.3, ... on the other, so the probability changes gradually across the plane.

3. Combining two perceptrons

Key point: Two perceptrons give two probability maps; adding them and passing the sum through a sigmoid gives a new map whose 0.5 boundary is curved.

3.1 The idea: superimpose, then smooth

Key point: Lay the two maps on top of each other, then smooth the result back into a probability.

Suppose the green points sit in a wedge, as in Figure 1. Perceptron 1 (left) separates them from the points below-left; perceptron 2 (middle) separates them from the points above-left. Neither line alone works: perceptron 1 misclassifies 44 of the 160 points, perceptron 2 misclassifies 40.

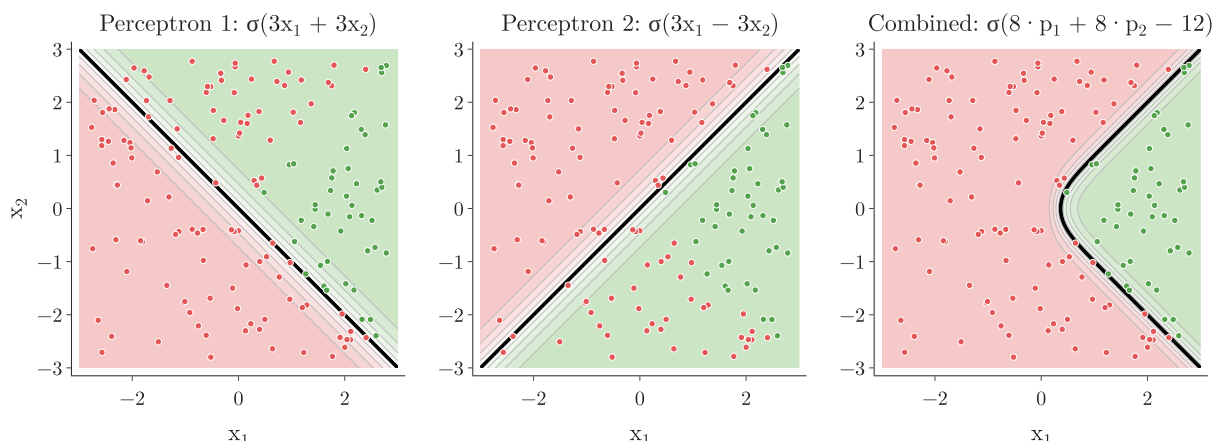


Figure 1: Two perceptrons and their combination. Shading: probability of green; black: the 0.5 boundary. The weights are our own, chosen to show the effect.

If we could lay the two maps on top of each other and smooth the result, we would get the right panel: a boundary that follows perceptron 2's line at the top, perceptron 1's line at the bottom, and bends round between them. The next two sections show the arithmetic that does this.

3.2 Adding the probabilities

Key point: For each point, add the two perceptrons' probabilities and pass the sum through a sigmoid, so the result is again a probability.

Take one student. Perceptron 1 says the probability of placement is 0.7, perceptron 2 says 0.8. Adding them gives 1.5, which cannot be a probability. The sigmoid fixes that:

1. **In words:** add the two probabilities, then squeeze the sum back into the range 0 to 1 with the sigmoid.

2. **Formula:**

$$\hat{y} = \sigma(p_1 + p_2)$$

3. **Example:** with $p_1 = 0.7$ and $p_2 = 0.8$,

$$\hat{y} = \sigma(1.5) = \frac{1}{1 + e^{-1.5}} = 0.818$$

We repeat this for every point. Adding is the superimposing; the sigmoid is the smoothing.

3.3 Weights and a bias

Key point: A weighted sum plus a bias lets one perceptron count more than the other, and lets us shift the result.

Plain adding treats both perceptrons equally. To make perceptron 1 count twice as much, we multiply its output by a weight of 10 and perceptron 2's by 5, and add a bias of 3:

1. **In words:** multiply each perceptron's probability by its weight, add the bias, and take the sigmoid.
2. **Formula:**

$$\hat{y} = \sigma(w_1 p_1 + w_2 p_2 + b)$$

3. **Example:** with $w_1 = 10$, $w_2 = 5$, $b = 3$:

$$z = 10 \times 0.7 + 5 \times 0.8 + 3 = 14, \quad \hat{y} = \sigma(14) = 0.99999917 \approx 1.000$$

Figure 1 uses this rule with $w_1 = w_2 = 8$ and $b = -12$. The bias -12 is what makes the combination demand **both** probabilities to be high:

Point (x_1, x_2)	p_1	p_2	$z = 8p_1 + 8p_2 - 12$	$\hat{y}$
(1, 0)	0.953	0.953	3.24	0.962
(0, 0)	0.5	0.5	-4.00	0.018

The point (0, 0) lies on both lines, so each perceptron alone gives it 0.5. The combination gives it 0.018, firmly red: its boundary has bent away from both lines, which is the rounded tip in Figure 1.

3.4 The combiner is a perceptron

Key point: Weighted sum, bias, sigmoid: the combining step is itself a perceptron, whose inputs are the outputs of perceptrons 1 and 2.

Look again at $\sigma(w_1 p_1 + w_2 p_2 + b)$. This expression is exactly what a perceptron computes, except that its inputs are not features of the data but the outputs of two other perceptrons. So the whole model is three perceptrons connected together (Figure 2).

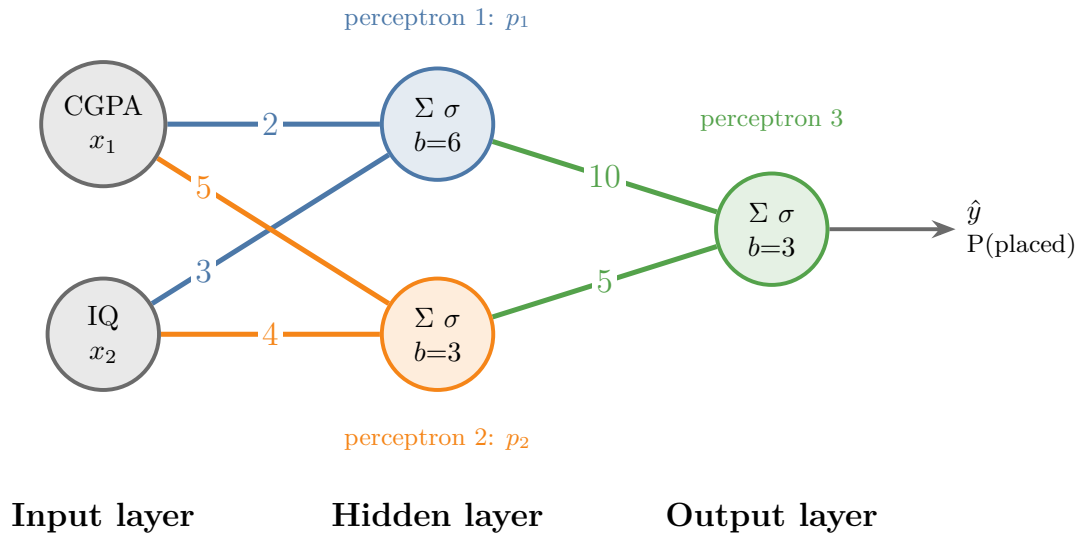


Figure 2: The model of Section 3.3 as a network: perceptrons 1 and 2 read CGPA and IQ; perceptron 3 combines their outputs

- Perceptron 1 computes $p_1 = \sigma(2x_1 + 3x_2 + 6)$, perceptron 2 computes $p_2 = \sigma(5x_1 + 4x_2 + 3)$.
- Both read the same two inputs, so CGPA and IQ each connect to both of them.
- Perceptron 3 computes $\hat{y} = \sigma(10p_1 + 5p_2 + 3)$.

The nodes now sit in layers: an **input layer** (CGPA, IQ), a **hidden layer** (perceptrons 1 and 2) and an **output layer** (perceptron 3). This layered network is a multi-layer perceptron. The idea behind it is short: a weighted (linear) combination of perceptrons, passed through a sigmoid, captures non-linearity that no single perceptron can.

Extra: The sigmoid between the layers is essential. Without it, the hidden layer would output $2x_1 + 3x_2 + 6$ and $5x_1 + 4x_2 + 3$, and the output layer would add multiples of them: still a straight line. The [matrix multiplication Note](#) (section 7.2) shows the general version: stacked linear layers collapse into one.

4. Four ways to change the architecture

Key point: We can add hidden nodes, input nodes, output nodes or hidden layers. Each answers a different need.

The **architecture** of a neural network is how its nodes are arranged and connected: how many layers, how many nodes in each, and which nodes are joined by weights. Figure 3 shows four ways to change it, with the changed part in red.

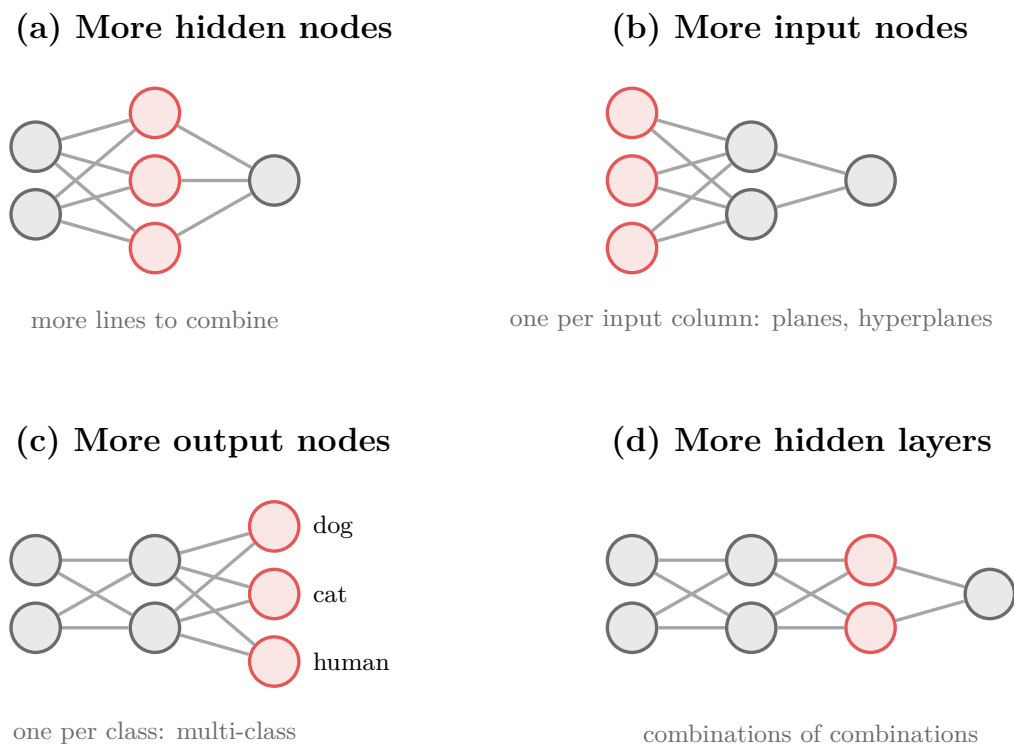


Figure 3: Four ways to change an MLP's architecture; the changed layer is red

4.1 More nodes in a hidden layer

Key point: Each extra hidden node adds one more line to combine, so the boundary can bend in more places.

With three hidden perceptrons instead of two (Figure 3a), the output perceptron combines three lines. The formula just gains a term. For hidden outputs 0.2, 0.3 and 0.4, weights 1, 2 and 3 and a bias of 10:

$$z = 1 \times 0.2 + 2 \times 0.3 + 3 \times 0.4 + 10 = 12.0, \quad \hat{y} = \sigma(12.0) = 0.99999$$

More hidden nodes let the network fit data with more non-linearity.

4.2 More nodes in the input layer

Key point: One input node per feature; with three features, each hidden perceptron is a plane instead of a line.

The input layer has one node per feature, so we add input nodes only when the data gains a feature (Figure 3b). With 12th marks as a third feature, every point lives in 3D. Each hidden perceptron is then a plane, and the output perceptron combines planes in exactly the

same way. With four or more features the planes become hyperplanes (see the [equation of a hyperplane Note](#)).

4.3 More nodes in the output layer

Key point: For multi-class classification, one output node per class; the class with the highest output wins.

So far the output layer had one node. To tell whether a photo shows a dog, a cat or a human, we give the output layer three nodes, one per class (Figure 3c). Each gives a score for its class, and we predict the class with the highest one.

Extra: In practice the three output scores are turned into probabilities that add up to 1 with the softmax function, exactly as in [softmax regression](#) (Goodfellow et al. 2016, §6.2.2.3). Softmax with categorical cross-entropy is the combination from the [perceptron loss Note](#).

4.4 More hidden layers

Key point: A second hidden layer combines the curved boundaries of the first, so the network can draw even more complex shapes.

We can also add whole hidden layers (Figure 3d). The first hidden layer still draws straight lines. The second combines those into curves, the third combines curves into more complex shapes, and so on.

With enough hidden nodes, an MLP can approximate any continuous function: the **universal approximation theorem** (see the [types of neural networks Note](#)). The theorem only says such weights exist; it does not promise that training finds them (Goodfellow et al. 2016, §6.4.1). The price is more parameters and longer training.

5. Seeing it work

Key point: Two hidden nodes are enough for XOR. Each draws one straight line, and the output node keeps the strip between the two lines.

5.1 XOR with two hidden nodes

Key point: Hidden node 1 and hidden node 2 each draw a line; the output node says “class 1” only between them. All 200 points are classified correctly.

We return to XOR, the data no single perceptron can separate (see the [problem with the perceptron Note](#)). Each of the four XOR inputs, centred as $(\pm 1.5, \pm 1.5)$, gets a small cloud of 50 noisy points. Class 1 (green) is where exactly one input is positive, as in the XOR table.

The network has 2 inputs, **2 sigmoid hidden nodes** and 1 sigmoid output node. We train it with plain [gradient descent](#) on the [log loss](#): learning rate 1, 5,000 steps, starting from

small random weights. (How the gradients of the hidden weights are found is the topic of the [backpropagation Notes](#).)

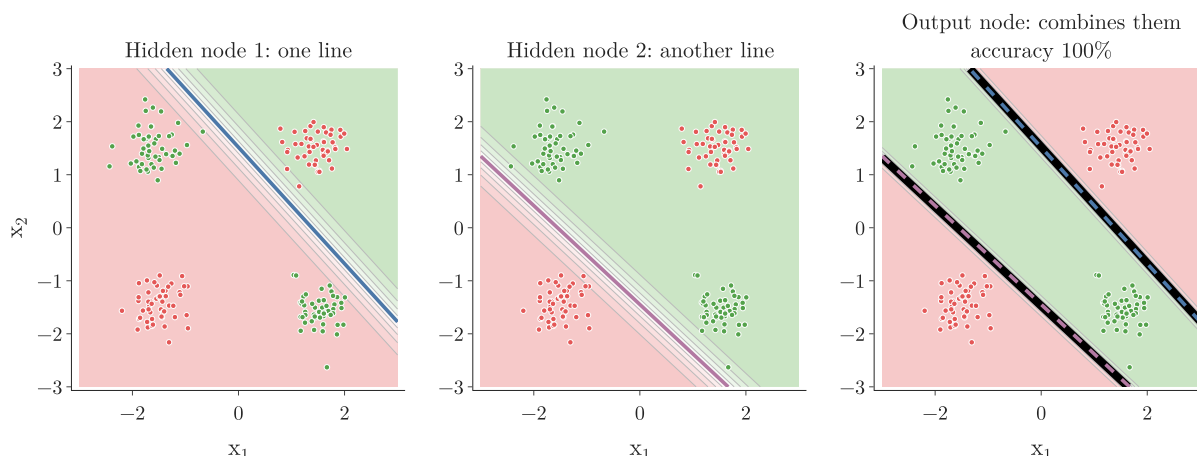


Figure 4: XOR with two hidden nodes. Left and middle: what each hidden node learned, with its line (the 0.5 boundary). Right: the output node’s map; its boundary (black) lies on the two hidden lines (dashed).

Figure 4 reads like Section 3, from left to right:

- **Hidden node 1** draws a line just below the top-right cluster: it says 1 only for the $(+, +)$ cluster.
- **Hidden node 2** draws a parallel line just above the bottom-left cluster: it says 1 for every cluster except $(-, -)$.
- **The output node** computes $\sigma(-12.6 h_1 + 12.2 h_2 - 5.8)$: class 1 when node 2 is on **and** node 1 is off. The region where both hold is exactly the strip between the two lines, where the two green clusters sit.

The accuracy is 100%. The same training from 100 different random starting weights also reached 100% every time (Notebook).

The trained network matches the textbook solution. Goodfellow et al. (§6.1) write down a network with two hidden units that solves the four XOR points (0 or 1 inputs) exactly. Its two units bend at two parallel lines, $x_1 + x_2 = 0$ and $x_1 + x_2 = 1$, and the output is $h_1 - 2h_2$. Working the four points through gives 0, 1, 1, 0: the output is high only along the strip around $x_1 + x_2 = 1$, where the two class-1 points sit, and drops back to 0 on either side.

Extra: Our hidden nodes use the sigmoid, as everywhere in this Note. Goodfellow et al. use ReLU, an activation taught later. The picture is the same: two lines, and the output keeps the strip between them.

5.2 Other shapes

Key point: On circles and spirals, small MLPs find the curved boundaries a single perceptron cannot. A poor setup still fails.

TensorFlow Playground (see the [problem with the perceptron Note](#)) lets us build small networks in the browser and watch them train. Figure 5 repeats two more of its demos in Python, with scikit-learn’s `MLPClassifier`, so the results can be rerun from the Notebook.

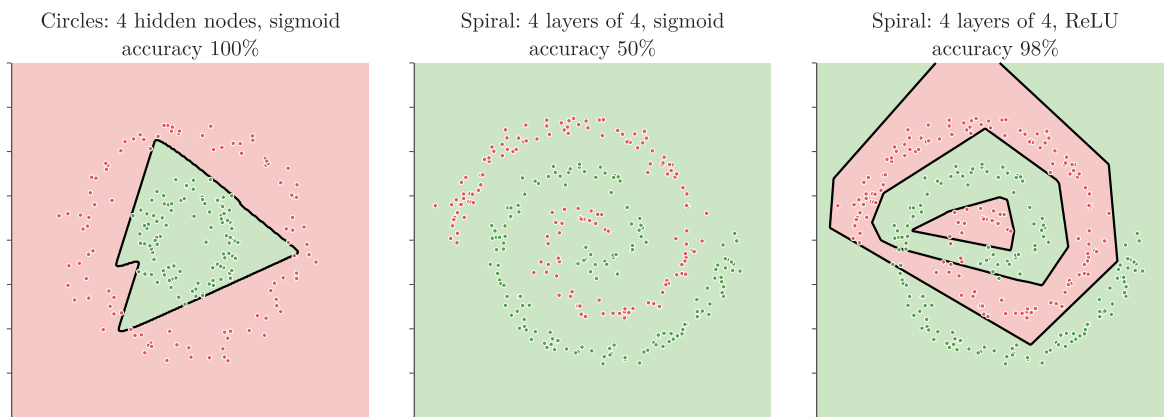


Figure 5: MLPs trained on two circles and on two spirals

Reading the panels in order:

- **Circles, 4 hidden nodes:** 100%. Four lines combine into a closed shape around the inner circle. With fewer nodes it falls short: 66% with 1 node, 85% with 2, 99% with 3 (Notebook).
- **Spirals, 4 hidden layers of 4 sigmoid nodes:** 50%, no better than guessing. The network predicts one class everywhere.
- **Spirals, same layers with ReLU:** 98% in the figure. Changing only the activation function makes the deep network trainable.

Averaged over 5 random starts (adam solver, Notebook), the sigmoid network stays at 50% every time, while the ReLU network reaches 84% (73% to 98%). The sigmoid stalls like a message passed down a long line of whisperers, each one quieter than the last; the [vanishing and exploding gradients Note](#) shows why.

TensorFlow Playground also draws what each hidden node has learned. A first-layer node computes $\sigma(w_1x_1 + w_2x_2 + b)$, so its 0.5 boundary is always a straight line; the later layers combine those lines into curves, as in Figure 4.

Python: An MLP in scikit-learn.

```
from sklearn.neural_network import MLPClassifier
```

```
# one hidden layer with 4 nodes, sigmoid activation
```

```
clf = MLPClassifier(hidden_layer_sizes=(4,),
                    activation="logistic",
                    solver="lbfgs", max_iter=5000,
                    random_state=0)

clf.fit(X, y)
clf.score(X, y)      # 1.0 on the circles data
hidden_layer_sizes=(4, 4, 4, 4) gives four hidden layers of 4 nodes. scikit-learn
calls the sigmoid "logistic". The input and output layers are sized automatically from
X and y.
```

Extra: ReLU is another activation function, the recommended default for hidden units in modern networks (Goodfellow et al. 2016, §6.3). The **solver** (lbfgs or adam) is the method used to find the weights; adam is one of the optimizers covered later. The circles use lbfgs and the spirals adam, because each worked best on its data. Averaged over 5 random starts (Notebook): on the circles, lbfgs 97% against adam 47%; on the ReLU spirals, adam 84% against lbfgs 78%.

6. Summary

Change	What it adds	When
More hidden nodes	More lines to combine	Data with more non-linearity
More input nodes	Planes or hyperplanes instead of lines	Data with more features
More output nodes	One output per class	Multi-class classification
More hidden layers	Combinations of combinations	Very complex boundaries

- A sigmoid perceptron gives every point a probability; its 0.5 line is a straight boundary.
- Combining perceptrons: $\hat{y} = \sigma(w_1p_1 + w_2p_2 + b)$, itself a perceptron.
- Input layer, hidden layer, output layer: that is a multi-layer perceptron.
- The sigmoid between layers is what lets the combined boundary curve.
- Two hidden nodes already solve XOR: two lines, and the output keeps the strip between them.
- With enough nodes and layers an MLP can approximate any function; in practice the setup (size, activation, solver) decides whether training finds it.

7. Sources

- Goodfellow, Bengio and Courville, *Deep Learning*, MIT Press, 2016, §6.1 (a network with two hidden units that solves XOR), §6.2.2.3 (softmax output units), §6.3 (ReLU as the default hidden unit), §6.4.1 (universal approximation: the weights exist, but training may not find them).

8. Key terms

Term	Meaning
Combining perceptrons	Feeding several perceptrons' outputs, weighted and with a bias, into another perceptron
Architecture (of a neural network)	How the nodes are arranged in layers and connected by weights
Multi-class output layer	An output layer with one node per class; the highest output gives the prediction
MLPClassifier	scikit-learn's multi-layer perceptron for classification
hidden_layer_sizes	MLPClassifier setting: the number of nodes in each hidden layer, e.g. (4, 4)
ReLU	An activation function used in most deep networks, taught in a later Note